

AI-POWERED

Enterprise PPM Platform

Complete Developer Guide

HLD · LLD · Day-Wise Roadmap · Module Breakdown

9	30+	45	AI
Modules	API Endpoints	Dev Days	Powered

Python 3.12 | FastAPI | Next.js 14 | PostgreSQL 16 | SQLAlchemy | JWT Auth | OpenAI API | Docker

■ Table of Contents

1	Project Overview & Vision	3
2	High-Level Design (HLD)	4
3	Low-Level Design (LLD)	7
3.1	Database Schema	7
3.2	API Endpoint Reference	9
3.3	Module Architecture	11
4	Week-by-Week Development Roadmap	12
4.1	Week 1 – Foundation (Day 1 DONE ✓)	12
4.2	Week 2 – Authentication Module	13
4.3	Week 3 – Portfolio & Project Module	15
4.4	Week 4 – Task & Resource Module	17
4.5	Week 5 – Budget & Risk Module	19
4.6	Week 6 – Dashboard & Analytics	21
4.7	Week 7 – AI Engine (Core)	23
4.8	Week 8 – Frontend UI	25
4.9	Week 9 – Integration & Testing	27
5	Deployment Guide	29
6	Quick Reference Cheat Sheet	30

1. Project Overview & Vision

What Are You Building?

The AI-Powered Enterprise PPM (Project Portfolio Management) platform is a full-stack, enterprise-grade web application that replaces the fragmented tool ecosystem of Microsoft Project + Jira + Monday.com + Excel with a single, AI-driven command centre. It helps organisations manage multiple projects, teams, budgets, and risks from one place — with AI predicting problems before they happen.

Problems Being Solved

Project Visibility	Managers cannot instantly see which projects are on-track or at risk.
Resource Management	Employees are over/under-allocated with no real-time visibility.
Budget Control	Budget overruns are discovered too late.
Risk Management	Risks tracked in scattered Excel sheets, found too late.
Decision Making	Executives lack data to decide which projects to fund or stop.

9 Core Modules

#	Module	Purpose
1	Auth & User Management	Login, JWT, Role-Based Access Control
2	Portfolio Management	Group and track collections of projects
3	Project Management	CRUD for projects with status and dates
4	Task Management	Tasks per project with assignees
5	Resource Management	Track employee allocations across projects
6	Budget Management	Planned vs actual cost tracking
7	Risk Management	Risk registry with impact and mitigation
8	Dashboard & Analytics	Charts, KPIs, executive views
9	AI Engine	Risk prediction, delay alerts, summaries, chat

2. High-Level Design (HLD)

The HLD defines the overall system architecture — how the major components interact at a high level, without going into code details.

2.1 System Architecture Diagram

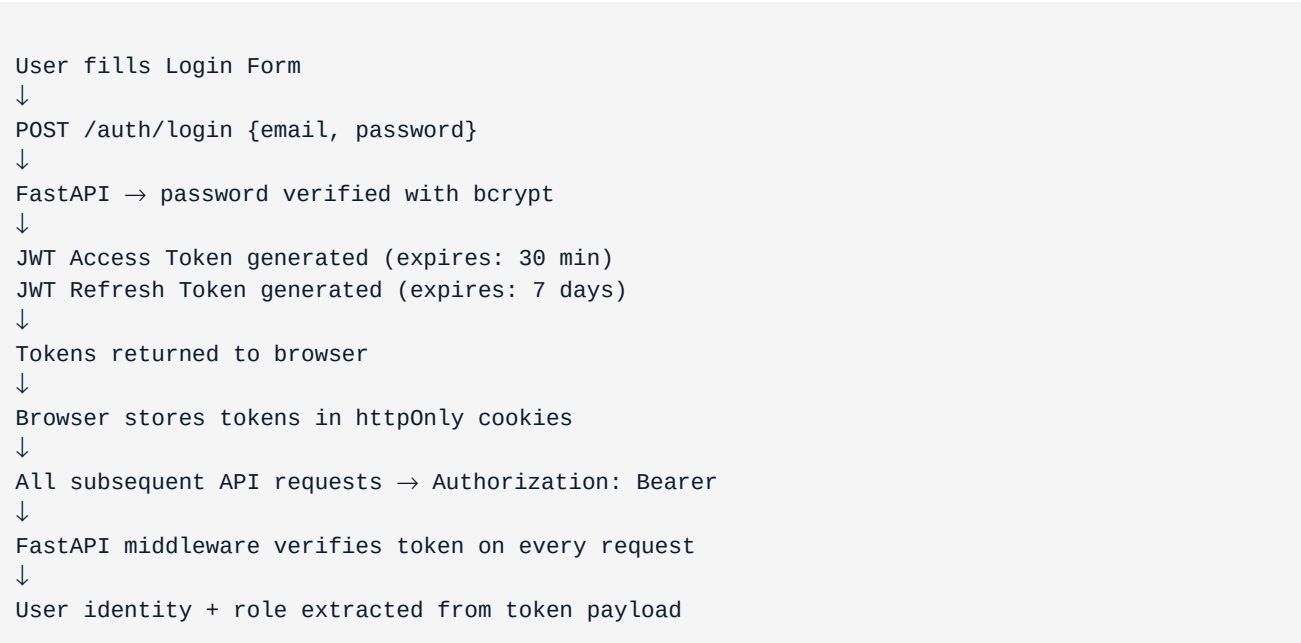


2.2 Technology Stack

Layer	Technology	Why / Purpose
Frontend Framework	Next.js 14	React-based, TypeScript, App Router, SSR/SSG
UI Library	shadcn/ui + Tailwind CSS	Enterprise component library

Layer	Technology	Why / Purpose
HTTP Client	Axios	REST API calls from frontend
Charts	Recharts	Dashboard visualisations
Backend Framework	FastAPI	High-performance Python REST API
ASGI Server	Uvicorn	Runs FastAPI asynchronously
ORM	SQLAlchemy 2.0	Python to SQL mapping
Migrations	Alembic	Schema version control
Database	PostgreSQL 16	Relational database
Authentication	JWT (python-jose)	Stateless token auth
Password Hashing	bcrypt (passlib)	Secure password storage
AI Framework	LangChain	AI agent and RAG workflows
LLM	OpenAI GPT-4o	Text generation and analysis
Containerisation	Docker + Docker Compose	Deployment packaging
Reverse Proxy	Nginx	Traffic routing and SSL termination
Version Control	Git + GitHub	Code management

2.3 Authentication Flow



2.4 Role-Based Access Control (RBAC)

Role	Permissions
Admin	Full access to everything including user management

Role	Permissions
Portfolio Manager	Create/manage portfolios, view all projects
Project Manager	Manage own projects, tasks, and resources
Team Member	View assigned tasks, update task status only

2.5 Data Flow – Creating a Project

Step-by-step flow of a typical API operation:

1. PM fills 'New Project' form in Next.js frontend
2. Frontend calls POST /api/projects with JWT in header
3. Nginx forwards request to FastAPI backend
4. FastAPI middleware validates JWT, extracts user_id and role
5. Router receives request, calls project_service.create_project()
6. Service validates input using Pydantic schema
7. SQLAlchemy creates INSERT query, saves to PostgreSQL
8. Response: Project object returned as JSON
9. Frontend receives response, updates UI state, redirects to project page

3. Low-Level Design (LLD)

3.1 Database Schema

All tables listed below with every column, type, and relationship.

Table: users

Column	Type / Constraint	Notes
id	SERIAL PRIMARY KEY	Auto-increment user ID
email	VARCHAR(255) UNIQUE NOT NULL	Login email
hashed_password	VARCHAR(255) NOT NULL	bcrypt hashed
full_name	VARCHAR(100) NOT NULL	Display name
role	ENUM('admin','portfolio_mgr','project_mgr','member')	RBAC role
is_active	BOOLEAN DEFAULT TRUE	Soft disable
created_at	TIMESTAMP DEFAULT NOW()	Record created
updated_at	TIMESTAMP DEFAULT NOW()	Last updated

Table: portfolios

Column	Type / Constraint	Notes
id	SERIAL PRIMARY KEY	
name	VARCHAR(200) NOT NULL	Portfolio name
description	TEXT	Optional description
owner_id	INT FK → users.id	Portfolio manager
status	VARCHAR(50) DEFAULT 'active'	active / archived
created_at	TIMESTAMP DEFAULT NOW()	

Table: projects

Column	Type / Constraint	Notes
id	SERIAL PRIMARY KEY	
portfolio_id	INT FK → portfolios.id	Parent portfolio
name	VARCHAR(200) NOT NULL	Project name
description	TEXT	

Column	Type / Constraint	Notes
status	ENUM('planning','in_progress','on_hold','completed','cancelled')	
start_date	DATE NOT NULL	
end_date	DATE NOT NULL	
budget_planned	NUMERIC(15,2) DEFAULT 0	In INR/USD
manager_id	INT FK → users.id	Project manager
health_score	INT DEFAULT NULL	AI-computed 0-100
created_at	TIMESTAMP DEFAULT NOW()	

Table: tasks

Column	Type / Constraint	Notes
id	SERIAL PRIMARY KEY	
project_id	INT FK → projects.id	
title	VARCHAR(300) NOT NULL	
description	TEXT	
status	ENUM('todo','in_progress','review','done','blocked')	
priority	ENUM('low','medium','high','critical')	
assigned_to	INT FK → users.id	
due_date	DATE	
estimated_hours	NUMERIC(6,2)	
actual_hours	NUMERIC(6,2) DEFAULT 0	
created_at	TIMESTAMP DEFAULT NOW()	

Table: resources

Column	Type / Constraint	Notes
id	SERIAL PRIMARY KEY	
user_id	INT FK → users.id UNIQUE	One resource per user
skills	TEXT[]	Array of skill tags
hourly_rate	NUMERIC(10,2)	Cost per hour
availability_pct	INT DEFAULT 100	Available capacity %

Table: allocations

Column	Type / Constraint	Notes
id	SERIAL PRIMARY KEY	
resource_id	INT FK → resources.id	
project_id	INT FK → projects.id	
allocation_pct	INT NOT NULL	e.g. 50 means 50%
start_date	DATE NOT NULL	
end_date	DATE NOT NULL	

Table: budgets

Column	Type / Constraint	Notes
id	SERIAL PRIMARY KEY	
project_id	INT FK → projects.id UNIQUE	
planned_cost	NUMERIC(15,2) DEFAULT 0	
actual_cost	NUMERIC(15,2) DEFAULT 0	
remaining	NUMERIC(15,2) GENERATED ALWAYS AS (planned_cost - actual_cost) STORED	Auto-calc
last_updated	TIMESTAMP DEFAULT NOW()	

Table: risks

Column	Type / Constraint	Notes
id	SERIAL PRIMARY KEY	
project_id	INT FK → projects.id	
description	TEXT NOT NULL	Risk description
impact	ENUM('low','medium','high','critical')	
probability	ENUM('low','medium','high')	
status	ENUM('open','mitigated','closed')	
mitigation_plan	TEXT	
ai_predicted	BOOLEAN DEFAULT FALSE	Set by AI engine
created_at	TIMESTAMP DEFAULT NOW()	

3.2 API Endpoint Reference

All REST endpoints for the platform. Base URL: <http://localhost:8000/api/v1>

Auth /auth

Method	Path	Description	Access
POST	/register	Register new user	Public
POST	/login	Login, returns JWT tokens	Public
POST	/refresh	Refresh access token	Auth
POST	/logout	Invalidate refresh token	Auth
GET	/me	Get current user profile	Auth

Portfolios /portfolios

Method	Path	Description	Access
GET	/	List all portfolios (paginated)	PM+
POST	/	Create new portfolio	Admin/PM
GET	/{{id}}	Get portfolio detail + projects	PM+
PUT	/{{id}}	Update portfolio	Owner/Admin
DELETE	/{{id}}	Archive portfolio	Admin
GET	/{{id}}/summary	AI-generated portfolio summary	PM+

Projects /projects

Method	Path	Description	Access
GET	/	List projects (filter by portfolio, status)	Auth
POST	/	Create project	PM+
GET	/{{id}}	Get project detail	Auth
PUT	/{{id}}	Update project	Owner/Admin
DELETE	/{{id}}	Delete project	Admin
GET	/{{id}}/health	Get AI health score	Auth
GET	/{{id}}/risks	Get project risks	Auth
GET	/{{id}}/tasks	Get project tasks	Auth

Tasks /tasks

Method	Path	Description	Access
GET	/	List tasks (filter by project/assignee)	Auth
POST	/	Create task	PM
GET	/{{id}}	Get task detail	Auth
PUT	/{{id}}	Update task (incl. status)	Auth
DELETE	/{{id}}	Delete task	PM+
PUT	/{{id}}/assign	Assign/reassign task	PM+

Resources /resources

Method	Path	Description	Access
GET	/	List all resources with allocation	PM+
GET	/{{id}}	Get resource detail	Auth
PUT	/{{id}}	Update skills/rate	Admin
POST	/allocate	Allocate resource to project	PM+
GET	/overallocated	List resources > 100% allocation	PM+
GET	/available	List available resources for a period	PM+

Budgets /budgets

Method	Path	Description	Access
GET	/project/{{project_id}}	Get project budget detail	Auth
PUT	/project/{{project_id}}	Update actual cost	PM+
GET	/overruns	List projects with budget overruns	Admin/PM

Risks /risks

Method	Path	Description	Access
GET	/	List risks (filter by project)	Auth
POST	/	Create risk	PM+
PUT	/{{id}}	Update risk / mitigation	PM+
DELETE	/{{id}}	Delete risk	PM+
POST	/predict/{{project_id}}	AI-predict risks for project	PM+

Dashboard /dashboard

Method	Path	Description	Access
GET	/kpis	Portfolio-level KPIs	Auth
GET	/project-health	Health scores for all projects	Auth
GET	/budget-chart	Budget vs actual data for chart	Auth
GET	/resource-chart	Allocation % per resource	Auth
GET	/risk-matrix	Risk distribution matrix	Auth

AI Engine /ai

Method	Path	Description	Access
POST	/chat	Chat with AI about project data	Auth
GET	/summary/{project_id}	Generate project AI summary	Auth
GET	/predict-delay/{project_id}	Predict delay probability	PM+
GET	/recommendations	Portfolio-level AI recommendations	PM+

3.3 Module File Structure

Each module follows the same clean architecture pattern:

```

backend/
├── app/
│   ├── main.py ← FastAPI app + router registration
│   ├── database.py ← Engine, SessionLocal, Base
│   ├── dependencies.py ← get_db(), get_current_user()
│   └──
├── models/ ← SQLAlchemy ORM models (1 file per table)
│   ├── user.py
│   ├── portfolio.py
│   ├── project.py
│   ├── task.py
│   ├── resource.py
│   ├── budget.py
│   └── risk.py
├──
├── schemas/ ← Pydantic validation schemas
│   ├── user.py ← UserCreate, UserOut, UserUpdate
│   ├── project.py ← ProjectCreate, ProjectOut, ProjectUpdate
│   └── ...
├──
├── routers/ ← FastAPI route handlers (thin layer)
│   ├── auth.py
│   ├── portfolios.py
│   ├── projects.py
│   ├── tasks.py
│   ├── resources.py
│   ├── budgets.py
│   ├── risks.py
│   ├── dashboard.py
│   └── ai.py
├──
├── services/ ← Business logic (pure Python)
│   ├── auth_service.py
│   ├── project_service.py
│   ├── ai_service.py ← OpenAI + LangChain calls
│   └── ...
├──
├── core/
│   ├── config.py ← Settings (loads from .env)
│   ├── security.py ← JWT create/verify, password hash
│   └── middleware.py ← CORS, logging middleware
├──
├── utils/
│   ├── pagination.py
│   └── helpers.py
├──
├── alembic/ ← Database migration files
├── tests/ ← pytest test files
├── requirements.txt
└── .env
  
```

4. Week-by-Week Development Roadmap

You have completed Day 1 (Week 1). The roadmap below resumes from Day 2. Each day is designed for 2–4 hours of focused work. Beginners should not rush — understand each step before moving to the next.

WEEK 1 – Foundation

DAY 1

Environment Setup ■ COMPLETED

- ✓ VS Code + Extensions installed
- ✓ Python 3.12, Node.js, Git, PostgreSQL installed
- ✓ Project folder structure created
- ✓ FastAPI backend initialised (main.py running)
- ✓ Next.js frontend created (npm run dev working)
- ✓ PostgreSQL database created
- ✓ database.py with SQLAlchemy engine configured
- ✓ .env file with DATABASE_URL

DAY 2

Alembic Migrations + First Database Table

Goal: Set up Alembic and create the users table in PostgreSQL

1	Install Alembic	<code>pip install alembic</code>	Already in requirements if you followed Day 1
2	Initialise Alembic	<code>alembic init alembic</code>	Run inside backend/ folder
3	Edit alembic.ini	<code>sqlalchemy.url = postgresql://postgres:PASSWORD@localhost:5432/ppm_platform</code>	Replace PASSWORD
4	Edit alembic/env.py	<code>from app.models import Base target_metadata = Base.metadata</code>	Import your models Base
5	Create User model	Create backend/app/models/user.py <code>class User(Base): ...</code>	See code in next box
6	Create migration	<code>alembic revision --autogenerate -m "create users table"</code>	Generates migration file
7	Apply migration	<code>alembic upgrade head</code>	Creates table in PostgreSQL
8	Verify in pgAdmin	Open pgAdmin → ppm_platform → Tables → users	Should see users table

User Model Code (backend/app/models/user.py)

```
from sqlalchemy import Column, Integer, String, Boolean, Enum, DateTime
```

```

from sqlalchemy.sql import func
from app.database import Base

class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True, index=True)
    email = Column(String(255), unique=True, nullable=False, index=True)
    hashed_password = Column(String(255), nullable=False)
    full_name = Column(String(100), nullable=False)
    role = Column(Enum('admin', 'portfolio_mgr', 'project_mgr', 'member'),
        default='member', nullable=False)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), onupdate=func.now())

```

DAY 3**Core Config + Dependencies + CORS Setup****Goal: Create config.py, dependencies.py, and enable CORS**

- Create backend/app/core/config.py — load all environment variables using pydantic-settings
- Add to .env: SECRET_KEY, ALGORITHM=HS256, ACCESS_TOKEN_EXPIRE_MINUTES=30, REFRESH_TOKEN_EXPIRE_DAYS=7
- Create backend/app/core/security.py — functions: hash_password(), verify_password(), create_access_token(), decode_token()
- Create backend/app/dependencies.py — get_db() yields DB session, get_current_user() reads JWT and returns user
- Add CORS middleware to main.py — allow localhost:3000 (frontend)
- Test: uvicorn app.main:app --reload should still start cleanly

Security Functions (backend/app/core/security.py)

```

from passlib.context import CryptContext
from jose import jwt, JWTError
from datetime import datetime, timedelta
from app.core.config import settings

pwd_context = CryptContext(schemes=['bcrypt'], deprecated='auto')

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

def create_access_token(data: dict) -> str:
    expire = datetime.utcnow() + timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    data.update({'exp': expire})
    return jwt.encode(data, settings.SECRET_KEY, algorithm=settings.ALGORITHM)

```

DAY 4**Auth Module – Register API**

Goal: Build POST /auth/register endpoint

- Create backend/app/schemas/user.py — UserCreate(email, password, full_name, role), UserOut(id, email, full_name, role)
- Create backend/app/routers/auth.py — register() route function
- Create backend/app/services/auth_service.py — register_user() checks duplicate email, hashes password, saves to DB
- Register the router in main.py: app.include_router(auth_router, prefix='/api/v1/auth')
- Test in Postman: POST http://localhost:8000/api/v1/auth/register with JSON body
- Expected response: {"id": 1, "email": "...", "full_name": "...", "role": "member"}
- Check pgAdmin — user row should appear in users table

DAY 5**Auth Module – Login + JWT****Goal: Build POST /auth/login endpoint returning JWT tokens**

- Add login() route to auth.py — accepts email + password (OAuth2PasswordRequestForm)
- In auth_service.py — authenticate_user() fetches user from DB, verifies password
- On success: create_access_token({'sub': user.email, 'role': user.role})
- Return: {access_token, refresh_token, token_type: 'bearer'}
- Test in Postman: POST /auth/login with form-data (username=email, password=...)
- Copy the access_token — you will use it for all future API calls
- Add GET /auth/me route using get_current_user dependency — verify it works

DAY 6-7**Auth Module – Frontend Login Page****Goal: Build Login and Register pages in Next.js**

- Create frontend/src/app/login/page.tsx — Login form with email + password fields
- Create frontend/src/lib/api.ts — axios instance with base URL + JWT header injection
- On login form submit: call POST /auth/login API using axios
- Store JWT token in localStorage (development) or httpOnly cookie
- Create frontend/src/app/register/page.tsx — Registration form
- Add redirect to /dashboard on successful login
- Create a simple Protected Route wrapper — redirect to /login if no token
- Style both pages with Tailwind CSS and shadcn/ui Card, Input, Button components

WEEK 2 – Portfolio & Project Module

This week you build the two core management modules. By end of Week 2 you will be able to create portfolios containing projects.

DAY 8

Portfolio Model + Migration + CRUD API

- Create backend/app/models/portfolio.py — Portfolio model with all columns
- Run: alembic revision --autogenerate -m 'create portfolios table' then alembic upgrade head
- Create schemas/portfolio.py — PortfolioCreate, PortfolioOut, PortfolioUpdate
- Create services/portfolio_service.py — create, get_all, get_by_id, update, delete
- Create routers/portfolios.py — GET /, POST /, GET /{id}, PUT /{id}, DELETE /{id}
- Add RBAC: only admin and portfolio_mgr can create portfolios
- Register router in main.py with prefix='/api/v1/portfolios'
- Test ALL 5 endpoints in Postman (use JWT from login)

DAY 9

Project Model + Migration + CRUD API

- Create backend/app/models/project.py — Project model (includes portfolio_id FK)
- Run alembic migration for projects table
- Create schemas/project.py — ProjectCreate, ProjectOut, ProjectUpdate
- Create services/project_service.py — full CRUD logic
- Create routers/projects.py — 8 endpoints as per API reference
- Add filter: GET /projects?portfolio_id=1&status=in_progress
- Test all endpoints, verify FK relationship (project must belong to valid portfolio)

DAY 10-11

Portfolio + Project Frontend Pages

- Create frontend/src/app/dashboard/portfolios/page.tsx — List all portfolios as cards
- Create frontend/src/app/dashboard/portfolios/[id]/page.tsx — Portfolio detail with project list
- Create 'New Portfolio' modal — shadcn Dialog + Form
- Create frontend/src/app/dashboard/projects/page.tsx — Project list with filters
- Create frontend/src/app/dashboard/projects/[id]/page.tsx — Project detail page
- Create 'New Project' form with portfolio selector dropdown
- Add status badges (In Progress = blue, Completed = green, At Risk = red)
- Use Recharts for a simple project status doughnut chart on portfolio page

DAY 12-14

Task Module – Backend + Frontend

- Day 12: Create Task model + migration + CRUD API (6 endpoints)

- Day 12: Add status transitions: todo → in_progress → review → done
- Day 13: Create Task list page inside project detail — grouped by status (Kanban-style)
- Day 13: Create 'Add Task' form — title, description, assignee, priority, due date
- Day 14: Add drag-and-drop task status update (use react-beautiful-dnd or @dnd-kit)
- Day 14: Show overdue tasks with red highlighting
- Day 14: Filter tasks by assignee, priority, status
- Test: create project → add tasks → update status → verify in DB

WEEK 3 – Resource & Budget Module

DAY 15-16

Resource Management – Backend

- Create Resource model + Allocation model + run migrations
- Resource = linked to User (one resource per user)
- Allocation = resource assigned to project with percentage and date range
- API: POST /resources/allocate — assign resource to project
- API: GET /resources/overallocated — return resources where sum of allocations > 100%
- API: GET /resources/available — resources with capacity for new allocation
- Business rule: total allocation for one resource across all projects cannot exceed 100%
- Service logic: validate before saving, raise 400 if over-allocation would occur

DAY 17

Resource Management – Frontend

- Resource overview page — list all team members with their allocation %
- Visual allocation bar (0–100%) — red if > 80%, green if healthy
- Allocation modal — assign a resource to a project with date range
- Over-allocation warning banner — show alert if resource exceeds 100%
- Use Recharts stacked bar chart to show allocation per resource per project

DAY 18-19

Budget Module – Backend + Frontend

- Create Budget model (one per project) + migration
- API: GET /budgets/project/{id} — budget detail with planned, actual, remaining
- API: PUT /budgets/project/{id} — update actual_cost (called when expenses added)
- API: GET /budgets/overruns — list projects where actual > planned
- Frontend: Budget section inside project detail page
- Show: Planned Budget / Actual Spent / Remaining — as KPI cards
- Add Recharts bar chart: planned vs actual per project
- Color-code: remaining < 10% = red warning

DAY 20-21

Risk Module – Backend + Frontend

- Create Risk model + migration
- API: GET /risks?project_id=... — list risks for project
- API: POST /risks — create risk (description, impact, probability, mitigation_plan)
- API: PUT /risks/{id} — update risk status (open → mitigated → closed)

- Frontend: Risk Registry table inside project detail page
- Risk Matrix visualisation: 3x3 grid (Impact vs Probability)
- Color code: High+High = red, Low+Low = green
- Add 'Add Risk' form with all fields

WEEK 4 – Dashboard & Analytics

DAY 22-24

Dashboard API – All KPI Endpoints

- Create routers/dashboard.py + services/dashboard_service.py
- GET /dashboard/kpis — total projects, on-track count, at-risk count, delayed count, total budget, budget used %
- GET /dashboard/project-health — list of {project_name, health_score, status} sorted by health asc
- GET /dashboard/budget-chart — [{project, planned, actual}] for bar chart
- GET /dashboard/resource-chart — [{resource_name, allocation_%}] sorted desc
- GET /dashboard/risk-matrix — count of risks by impact+probability combination
- All endpoints require auth. Admin sees all, PM sees only their projects.
- Test each endpoint in Postman, verify aggregated numbers are correct

DAY 25-27

Dashboard Frontend – Executive View

- Create frontend/src/app/dashboard/page.tsx — main dashboard
- Top row: 4 KPI stat cards (Total Projects, At Risk, Budget Used, Resources Overallocated)
- Row 2: Recharts BarChart — Budget vs Actual for top 5 projects
- Row 3: Recharts LineChart — project health scores trend
- Row 4: Resource allocation HorizontalBarChart
- Row 5: Risk matrix 3x3 heat map (HTML+CSS grid, color-coded)
- Make dashboard responsive (works on mobile too)
- Add loading skeletons (shadcn Skeleton component) while data fetches

DAY 28

Navigation + Layout + UX Polish

- Create sidebar navigation component — links to Dashboard, Portfolios, Projects, Resources, Risks
- Add top header bar — user avatar, role badge, logout button
- Add breadcrumbs on all detail pages
- Add 404 page and error boundaries
- Make all tables sortable and searchable
- Add pagination on long lists

WEEK 5 – AI Engine (Core)

This is the most exciting and most complex week. You will integrate OpenAI's API and build the AI features that differentiate this platform. Take your time — understand each concept before coding.

DAY 29

OpenAI API Setup + First Integration Test

- Sign up at platform.openai.com and get your API key
- Add to .env: `OPENAI_API_KEY=sk-...`
- `pip install openai langchain langchain-openai`
- Create `backend/app/services/ai_service.py`
- Test: simple Python script to call GPT-4o and print response
- Test prompt: 'Summarise this project: name=Alpha, status=delayed, budget_used=90%, risks=3 open'
- Verify you get a useful summary back before building full features

DAY 30-31

AI Project Health Score

How the Health Score Works

The AI health score is calculated from 4 sub-scores, each 0-100, then weighted:

Component	Range	Weight	Calculation Logic
Schedule Health	0-100	25%	Days remaining vs planned duration ratio
Budget Health	0-100	25%	Remaining budget % (100% = not spent at all)
Resource Health	0-100	25%	Average allocation health of assigned team
Risk Health	0-100	25%	Inverse of open critical/high risks

- Create `calculate_health_score(project_id, db)` function in `ai_service.py`
- Fetch project, budget, tasks, risks from DB
- Calculate each sub-score with Python math (no AI needed for basic score)
- Store result in `projects.health_score` column (update on every API call)
- `GET /projects/{id}/health` returns the score + breakdown
- Optionally: send score data to GPT and ask it to explain in plain English
- Show score on project card with color: >75=green, 50-75=yellow, <50=red

DAY 32-33

AI Project Summary + Delay Prediction

- `GET /ai/summary/{project_id}` — fetch project data, format as prompt, call GPT-4o
- Prompt template: 'You are a senior PM. Given this project data, write a 3-sentence executive summary.'
- `GET /ai/predict-delay/{project_id}` — analyse tasks, resources, deadlines

- Prompt: 'Given these tasks and their statuses, what is the probability (%) of this project missing its deadline?'
- Ask GPT to return JSON: {probability: int, reasons: [str], recommendations: [str]}
- Parse JSON response and return structured data from API
- Show delay prediction on project detail page as a risk alert card

**DAY
34-35****AI Chat Assistant****Architecture: RAG-based Chat**

The chat assistant uses a Retrieval-Augmented Generation (RAG) pattern: instead of sending all data to GPT every time, you fetch only the relevant data based on the user's question, then include it in the prompt.

- POST /ai/chat — receives {message: str, conversation_history: [...]}
- Step 1: Analyse user message to determine intent (asking about projects, risks, budgets?)
- Step 2: Fetch relevant data from DB (e.g. if 'which projects are delayed' → fetch delayed projects)
- Step 3: Build system prompt: 'You are a PPM assistant. Here is the current data: {data}'
- Step 4: Send message + history + system prompt to GPT-4o
- Step 5: Return AI response as JSON
- Frontend: Create a chat widget (floating button → opens chat drawer)
- Chat stores conversation_history in React state, sends with each message
- Example questions to test: 'Which project has highest risk?', 'Show budget overruns', 'Predict completion date for Project Alpha'

WEEK 6 – Integration, Testing & Deployment

DAY
36-38

Writing Tests

Backend Tests (pytest)

- pip install pytest pytest-asyncio httpx
- Create backend/tests/test_auth.py — test register, login, token refresh
- Create backend/tests/test_projects.py — test CRUD, verify RBAC enforcement
- Create backend/tests/test_risks.py — test risk creation and status updates
- Run: pytest tests/ -v

Frontend Tests (Playwright E2E)

- npm install @playwright/test
- Write test: login flow → lands on dashboard
- Write test: create portfolio → create project inside it
- Write test: add task → change status → verify kanban updates
- Run: npx playwright test

DAY
39-41

Docker + Docker Compose

- Create backend/Dockerfile — FROM python:3.12, COPY, pip install, CMD uvicorn
- Create frontend/Dockerfile — FROM node:22, npm install, npm run build, nginx serve
- Create docker-compose.yml — 3 services: backend, frontend, db (postgres:16)
- Add nginx/nginx.conf — proxy /api → backend:8000, serve frontend on port 80
- Run: docker-compose up --build
- Test full stack via http://localhost (no port numbers, nginx routes everything)
- Add .dockerignore files to exclude node_modules, venv, .env from image

DAY
42-45

Cloud Deployment (AWS/GCP/Railway)

Recommended: Railway.app (Easiest for Beginners)

- Go to railway.app → New Project → Deploy from GitHub
- Add PostgreSQL plugin from Railway dashboard
- Set environment variables: DATABASE_URL, SECRET_KEY, OPENAI_API_KEY
- Railway auto-deploys on every git push to main branch
- Get your public URL (e.g. ai-ppm.railway.app)

Alternative: AWS EC2 (More Control)

- Launch EC2 t3.small instance (Ubuntu 24.04)
- SSH in, install Docker + Docker Compose
- Clone your GitHub repo, set .env with production values
- Run: `docker-compose up -d`
- Set up Nginx + Let's Encrypt SSL for HTTPS
- Point your domain DNS to EC2 public IP

5. Deployment Checklist

Environment Variables

- SECRET_KEY — long random string (use: `python -c 'import secrets; print(secrets.token_hex(32))'`)
- DATABASE_URL — production PostgreSQL URL
- OPENAI_API_KEY — your OpenAI key
- ALLOWED_ORIGINS — your frontend domain

Security

- HTTPS enabled (Let's Encrypt via Certbot)
- Passwords hashed with bcrypt (never stored plain)
- JWT tokens short-lived (30 min access, 7 day refresh)
- CORS restricted to known frontend domains only
- SQL injection prevented by SQLAlchemy ORM (never raw SQL with user input)

Database

- Run alembic upgrade head on first deploy
- Set up automated daily backups (`pg_dump`)
- Use connection pooling in production (`pgBouncer` or `SQLAlchemy pool_size`)

Performance

- Add database indexes on frequently queried columns (`project.status`, `task.project_id`)
- Add Redis caching for dashboard KPIs (cache for 60 seconds)
- Use Next.js static generation for public pages

6. Quick Reference Cheat Sheet

Common Commands

Action	Command
Start backend	cd backend && uvicorn app.main:app --reload
Start frontend	cd frontend && npm run dev
Activate venv (Win)	backend\venv\Scripts\activate
Activate venv (Mac/Linux)	source backend/venv/bin/activate
Create migration	alembic revision --autogenerate -m 'description'
Apply migrations	alembic upgrade head
Rollback 1 step	alembic downgrade -1
Run tests	cd backend && pytest tests/ -v
Install Python pkg	pip install && pip freeze > requirements.txt
Install Node pkg	npm install
Build Docker	docker-compose up --build
Stop Docker	docker-compose down
View logs	docker-compose logs -f backend

Common Errors & Fixes

Error	Fix
ImportError: No module named X	Run: pip install X (with venv activated)
CORS error in browser	Add frontend URL to CORSMiddleware allowed_origins in main.py
JWT decode error	Check SECRET_KEY is same in .env and server restart done
Alembic: Target database is not up to date	Run: alembic upgrade head
PostgreSQL connection refused	Check PostgreSQL service is running (pgAdmin connects?)
Next.js 404 on API call	Check NEXT_PUBLIC_API_URL in frontend .env.local points to backend
SQLAlchemy DetachedInstanceError	Add db.refresh(obj) after commit or use db.expunge_all()
OpenAI RateLimitError	You hit API limit — add retry logic with exponential backoff

Weekly Progress Tracker

Week	Days	Focus	Tick Off Each Day
Week 1	Days 1-7	Foundation + Auth Backend	■ ■ ■ ■ ■ ■ ■
Week 2	Days 8-14	Portfolio + Project + Task	■ ■ ■ ■ ■ ■ ■
Week 3	Days 15-21	Resource + Budget + Risk	■ ■ ■ ■ ■ ■ ■
Week 4	Days 22-28	Dashboard + Analytics	■ ■ ■ ■ ■ ■ ■
Week 5	Days 29-35	AI Engine	■ ■ ■ ■ ■ ■ ■
Week 6	Days 36-45	Testing + Deployment	■ ■ ■ ■ ■ ■ ■ ■ ■ ■

Beginner Tips

- Always test backend in Postman before building frontend for it — saves hours of debugging
- Commit to Git every day: `git add . && git commit -m 'Day X: description'`
- Read the FastAPI docs when stuck: <https://fastapi.tiangolo.com>
- Read the SQLAlchemy docs for DB questions: <https://docs.sqlalchemy.org>
- When an error occurs, READ the full error message — it usually tells you the exact problem
- Never hardcode passwords or API keys — always use `.env` and add `.env` to `.gitignore`
- When a feature feels too hard, break it into the smallest possible step and do just that
- The AI features (Week 5) require working CRUD first — don't skip ahead

You are building a production-grade enterprise platform. This is hard — every professional developer struggles at times. Stay consistent, commit daily, and trust the process. By Day 45, you will have a fully functional AI-Powered PPM Platform that you can deploy, demonstrate, and be proud of. Keep going! ■